# LEGALVISION: A KNOWLEDGE-DRIVEN AI FRAMEWORK FOR LEGAL UNDERSTANDING AND TRUST

Project ID - 25-26J-127

Project Proposal Report

Maxwell L.Y.

B.Sc. (Hons) Degree in Information Technology Specialized in

Data Science

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

July 2026

# LEGALVISION: A KNOWLEDGE-DRIVEN AI FRAMEWORK FOR LEGAL UNDERSTANDING AND TRUST

Project ID - 25-26J-127

Project Proposal Report

Maxwell L.Y.

B.Sc. (Hons) Degree in Information Technology Specialized in

Data Science

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

July 2026

## Declaration

I declare that this is my own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the nonexclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

| Name | Student ID | Signature |
|---|---|---|
| Maxwell L.Y. | IT22344892 | |

The supervisor/s should certify the proposal report with the following declaration.

The above candidate is carrying out research for the undergraduate Dissertation under my supervision.

.....

Signature of the supervisor

2025/08/28

Date

Name: Ms. Vindhya Kalapuge

.....

Signature of the co-supervisor

2025/8/28

Date

Name: Ms. Adya Dissanayake

## Abstract

This proposal presents the Bias & Risk Evaluator, a focused component for automated first-pass analysis of property and land law agreements (purchase, rental, lease, and related documents). The system accepts raw text or document files, segments agreements into clause-level units, and applies a fine-tuned transformer-based multi-label classifier to detect (a) types of bias specific to property law clauses and (b) risk severity levels (Low / Moderate / High). Detected bias and risk outputs are combined and calibrated into a single trust_score (0.0–1.0), where 0.0 denotes high bias and/or risk and 1.0 denotes low bias and/or risk. Explanations are produced by merging model-derived importance signals with concise rule-based rationales so flagged clauses can be presented in-context with interactive, color-coded highlights and on-demand tooltip text.

Methodology includes curated dataset creation and annotation (clause-level labels for bias categories and severity), clause boundary detection, transformer fine-tuning with class-aware loss and calibration, and a human-in-the-loop review process for iterative improvement. Evaluation will combine quantitative metrics (precision, recall, F1 per label; calibration of trust_score) and qualitative legal expert assessment of usefulness and false-positive/negative costs. Anticipated results are reliable identification of high-severity biased clauses, well-calibrated trust_scores useful for triage, and measurable reviewer time savings. The proposal concludes with recommendations for deployment: keep automated outputs advisory (not determinative), maintain transparent audit logs, and enforce dataset governance and privacy safeguards.

**Keywords:** Property law, Clause-level bias, Risk classification, Trust score, Explainable NLP

## Table of Contents

## List of Figures



## List of Tables



## List of Abbreviations

NLP — Natural Language Processing

ML — Machine Learning

UI — User Interface

API — Application Programming Interface

RoBERTa — Robustly optimized BERT approach

# 1. Introduction

## 1.1 Background & Literature review

Automated analysis of legal contracts has become an active area of research and product development because it reduces review time, lowers costs, and improves access to legal scrutiny for non-experts. Large, expert-annotated datasets and task formulations (e.g., clause highlighting, clause classification, document-level inference) have helped define benchmark problems for contract review and enabled meaningful model comparisons. A prominent example is the Contract Understanding Atticus Dataset (CUAD), which provides 13,000+ expert annotations across commercial contracts and is intended explicitly to benchmark automatic identification of clauses important for human review.

Work on contract reasoning and evidence extraction has produced additional datasets and tasks (for example, ContractNLI), which frame contract review as document-level natural language inference and evidence-span identification. These tasks demonstrate that contracts present particular linguistic challenges (long context, nested exceptions, domain-specific phrases) that require task-aware segmentation and model architectures.

Research and applied systems have also targeted *unfair* or *abusive* clause detection. CLAUDETTE is a notable system and line of work that automatically detects potentially unfair clauses in online Terms of Service and similar consumer contracts, demonstrating that ML can successfully flag clauses that may disadvantage one party and can be coupled with explanation modules for end users. However, CLAUDETTE and similar efforts focus largely on consumer-facing TOS/consumer contracts rather than property/land agreements.

In terms of modeling approaches, transformer-based language models (BERT and domain-adapted variants such as LEGAL-BERT) are now the dominant technique for clause classification, extraction and related legal NLP tasks. Studies show that domain adaptation (further pre-training on legal corpora and/or fine-tuning with careful hyperparameter choices) yields meaningful gains over off-the-shelf models for legal tasks, and that transformer models produce the strongest baseline performance on clause identification, labeling and evidence extraction tasks.

Recent surveys and reviews in contract classification and legal NLP confirm two practical conclusions for systems that flag risky or biased clauses: (1) high-quality, domain-specific annotations are critical (CUAD and ContractNLI illustrate this), and (2) model outputs must be combined with explainability and human-in-the-loop review to be useful and ethically acceptable in legal contexts. These points motivate a pipeline that includes clause segmentation, transformer fine-tuning on domain labels, output calibration, and an explainable UI for review and remediation.

### 1.2 Research gap

From the literature and existing tools, we identify the following gaps that motivate this research:

1. **Domain specificity — property/land agreements:** Existing unfair-clause detectors (e.g., CLAUDETTE) concentrate on consumer Terms of Service or general commercial contracts. There is limited published work and few public datasets targeting clause bias and risk specifically in property agreements (leases, rentals, purchase/sale contracts). This limits models' ability to recognize the particular patterns and imbalance common in landlord/tenant or buyer/seller clauses.
2. **Clause-level bias directionality (party-favoring labels):** Many contract datasets and prior models focus on *whether* a clause is important or possibly unfair, but do not explicitly label the *direction* of bias (i.e., which contracting party the clause favors). For decision support in property transactions, identifying party-directional bias (owner vs buyer/tenant) is critical for actionable recommendations.
3. **Summary trust metric:** While clause-level warnings are useful, practitioners also need an aggregate, calibrated metric (a *trust_score*) to quickly triage documents and prioritize human review. There is limited prior work that standardizes such an interpretable, calibrated trust score combining multi-label outputs and severity assessments.
4. **Jurisdictional and linguistic adaptability:** Public datasets tend to be skewed toward U.S. and other common law contracts; there is a gap in demonstrating generalization to other jurisdictions (including Sri Lanka) and to agreement variants that use different legal terminology or structural conventions.
5. **Explainability + human-in-the-loop evaluation:** Although systems exist that detect unfair clauses, few studies present a complete pipeline that couples detection, an explainability layer tailored to legal reasoning, calibrated scoring, and measurement of *practical usefulness* through expert review and user studies.

These gaps show there is room to develop a property-focused bias & risk evaluator that labels bias direction, assigns calibrated severity, produces an aggregate trust score, and is evaluated in collaboration with legal experts across jurisdictions.

## 1.3 Research problem

This research will develop and evaluate a Bias & Risk Evaluator component for property and land agreements with the following concrete problem statements and research questions:

**Primary problem statement**
How can we automatically detect and quantify clause-level bias (which party is favoured) and associated risk severity in property/land agreements (purchase, lease, rental), and how can these outputs be combined into a well-calibrated, interpretable trust_score (0.0–1.0) that helps end users decide whether to sign or to negotiate revisions?

**Sub-questions and issues to address**

1. Taxonomy design: What taxonomy of bias and risk categories best captures owner vs buyer/tenant imbalance in property agreements (e.g., Financial Imbalance, Unilateral Termination, Excessive Liability, Repair/Maintenance Burden, Jurisdiction/Dispute clauses)?
2. Dataset & annotation: How to construct an annotated, clause-level dataset for property agreements (covering common international patterns and Sri Lankan specifics) with reliable inter-annotator agreement?
3. Segmentation & representation: What clause-segmentation strategy and contextual windowing (sentence, paragraph, clause span) yields the best evidence spans for bias detection in long documents?
4. Modeling & calibration: Which transformer variant (e.g., LEGAL-BERT, RoBERTa) and training regimen (fine-tuning with class weights, calibration techniques like temperature scaling) achieve reliable multi-label predictions for both bias direction and severity, and how should outputs be combined into a calibrated trust_score?
5. Explainability & UX: How to present model rationales and rules (attention-based highlights, short rule-based explanations) so that non-expert users (and legal reviewers) can understand, verify and act on the findings?
6. Evaluation & deployment risk: What quantitative (precision/recall/F1 per label; calibration error for trust_score) and qualitative (legal expert usefulness, false positive/negative impact) evaluation framework will demonstrate the component's utility while limiting harm from automation?

## 2. Objectives

### 2.1 Main objective

To design, implement and evaluate a Bias & Risk Evaluator for property and land agreements (purchase, rental, lease, and related documents) that automatically detects clause-level bias (which party is favoured), quantifies risk severity (Low / Moderate / High), and computes a calibrated trust_score (0.0–1.0) to support user decision-making and triage prior to signing.

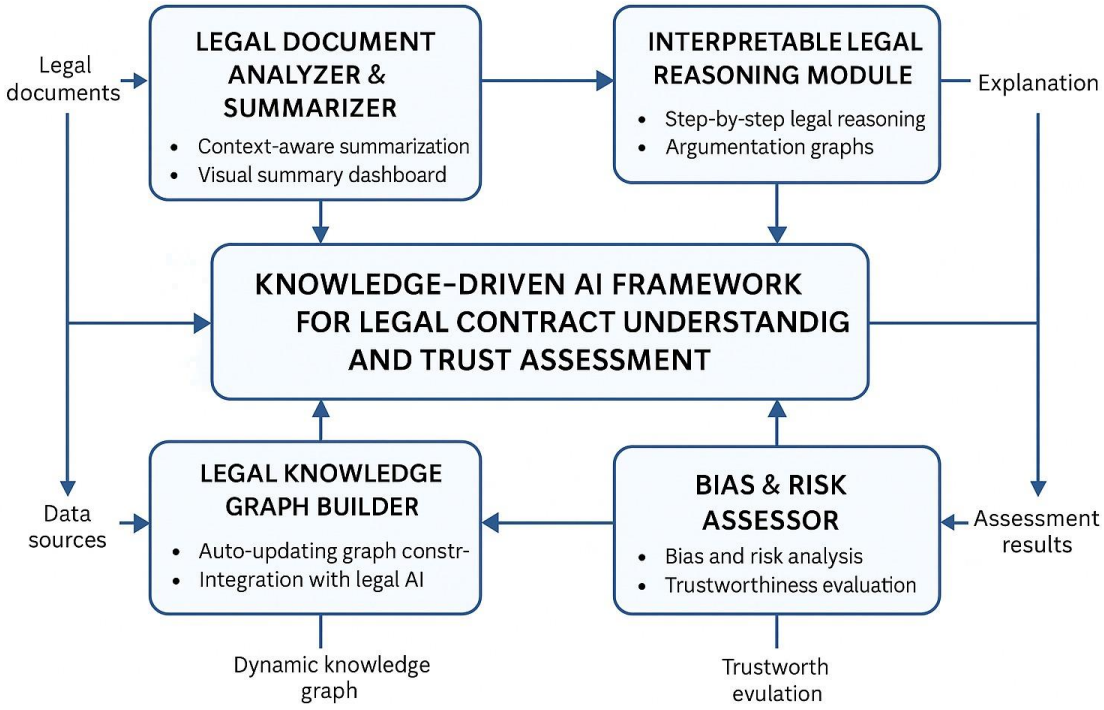
### 2.2 Specific objectives

1. Define a domain-specific taxonomy — design a clear set of bias and risk labels tailored to property agreements (e.g., Financial Imbalance, Unilateral Termination, Repair/Maintenance Burden, Jurisdiction/Dispute Bias) and label definitions for bias direction (Owner-favouring, Buyer/Tenant-favouring, Neutral).
2. Create and annotate a clause-level dataset — collect a representative corpus of property agreements (international + Sri Lanka examples), segment into clause units and annotate each clause for bias category, severity, and bias direction.
3. Develop robust clause segmentation & preprocessing — implement a reliable pipeline to extract textual clauses from DOCX/PDF/TXT (OCR where needed), normalize text, and identify clause boundaries and context windows for downstream classification.
4. Train and validate multi-label classifiers — fine-tune transformer-based models (e.g., Legal-BERT / RoBERTa variants) for multi-label classification of bias categories and severity, applying class-aware loss, calibration, and suitable data augmentation.
5. Design & calibrate the trust_score — define and validate an interpretable method to combine per-clause bias and severity outputs into a document-level trust_score in [0,1], and calibrate it so the score correlates with expert risk assessments.
6. Integrate explainability and UX — produce human-readable rationales for flagged clauses (attention/explainer highlights + concise rule-based notes) and implement an interactive UI that color-codes clauses by severity and exposes on-demand explanations.
7. Evaluate quantitatively and qualitatively — run a mixed evaluation: (a) quantitative metrics per label (precision, recall, F1, AUC where applicable) and calibration metrics for the trust_score; (b) qualitative assessment by legal experts measuring usefulness, time savings and cost of false positives/negatives.

# 3. Methodology

## 3.1 System architecture (high-level)

**Insert Figure 3.1: System Architecture Diagram here.**




(Recommended diagram parts: Document input → Text extraction & OCR → Clause segmentation → Preprocessing & NER → Multi-label classifier (bias categories + severity) → Explanation generator (attention + rule-based) → Trust-score calculator → UI (interactive highlights + reports) → Logging / Audit / Human-in-the-loop feedback loop)

Short description of components and how they fit together:

1. **Document Input**
    - Accepts DOCX / PDF / TXT / copy-paste.
2. **Text extraction & OCR**
    - Convert uploaded files to clean text with offsets for mapping highlights.
    - Tools: Apache Tika / pdfminer / Tesseract (for scanned documents).

3. **Clause segmentation & normalization**
    - Rule-based + ML hybrid: sentence/special-character heuristics (semicolons, numbered clauses) + a boundary detection model (e.g., Bi-LSTM/transformer on sentence boundaries) to produce clause spans and contextual windows (preceding/following clause).
4. **Preprocessing & NER**
    - Normalize dates, money amounts; extract named entities (parties, dates, amounts) to provide context features to the classifier.
5. **Bias & Risk classifier (multi-label)**
    - Transformer-based encoder fine-tuned for multi-label classification. Outputs per-clause: {bias_category(s), severity (Low/Moderate/High), bias_direction (Owner / Buyer/Tenant / Neutral), confidence scores}.
6. **Explainability / Rationale engine**
    - Combine model explainers (attention / SHAP / integrated gradients) with rule-based patterns to produce short, human-readable reasons to show in tooltips.
7. **Trust_score calculator**
    - Aggregates per-clause outputs into a single calibrated score in [0,1] that reflects document-level overall bias/risk (0 = high risk/bias; 1 = low risk/bias). Includes calibration/temperature scaling and optional weighting by clause importance.
8. **UI & Reporting**
    - Interactive viewer with color-coded highlights (yellow/orange/red), tooltips/sidebars with rationale and suggested remediation language, and export (annotated DOCX / summary CSV / PDF).

### 3.2 Tasks and sub-tasks

**Phase A — Design & preparation**

- A1. Finalize taxonomy of bias & risk labels (deliver annotation guidelines).

- A2. Define trust_score formula and calibration approach.
- A3. Prepare annotation platform & data pipeline scaffold (Label-studio / Doccano).

**Phase B — Data collection & annotation**

- B1. Collect documents: public leases/purchase agreements, templates, vendor-supplied redacted contracts, and Sri Lanka-specific examples.
- B2. Preprocess & OCR where necessary, run automatic clause segmentation.
- B3. Pilot annotation: annotate a seed set (200–500 clauses), compute inter-annotator agreement (Cohen's κ), refine guidelines.
- B4. Full annotation: expand to target dataset size (e.g., 5,000–15,000 clauses depending on scope).

**Phase C — Modeling & tooling**

- C1. Baseline: simple classifiers (logistic, TF-IDF) to set baselines.
- C2. Fine-tune transformer model(s) (Legal-BERT, RoBERTa variants) for multi-label classification.
- C3. Calibration: apply temperature scaling / isotonic regression for confidence calibration and tune trust_score mapping.
- C4. Integrate NER and rule-based heuristics for hybrid explainers.

**Phase D — UI & integration**

- D1. Build interactive viewer prototype (web app: React / Flask).
- D2. Integrate model inference API and highlight mapping (maintain character offsets).
- D3. Implement export features and logging.

**Phase E — Evaluation & iteration**

- E1. Quantitative evaluation: precision/recall/F1 per label, macro/micro averages, calibration error (ECE) for trust_score.
- E2. Qualitative evaluation: legal-expert review (usability study, time-savings measurement, perceived trust).

- E3. Iterate models guided by expert feedback and error analysis.

**Phase F — Reporting & deployment guidance**

- F1. Document results, produce user & deployment documentation.
- F2. Ethics & governance appendix (data handling, limitations, human-in-the-loop recommendations).
- F3. Final prototype handover and presentation.

### 3.3 Materials, equipment and software

**Hardware**

- Development machines (laptops/desktops).
- GPU access for training: university GPU nodes, cloud (GCP/AWS/Azure) with at least one GPU (e.g., NVIDIA T4/V100) for experiments.
- Optional: local server for inference testing.

**Software / Libraries**

- Python 3.x, PyTorch and/or TensorFlow, HuggingFace transformers.
- OCR & text extraction: Tesseract, Apache Tika, pdfminer.
- Annotation tools: Label-studio or Doccano.
- Explainability: SHAP, Captum (or similar).
- Web UI: React (frontend), Flask/FastAPI (backend) or Next.js.
- Storage & versioning: Git, cloud storage (GCS/S3), MLflow/Weights & Biases for experiments.
- Office software: MS Word for final report and DOCX export testing.

**Other materials**

- Legal expert time for annotation and qualitative evaluation (hourly or in-kind).
- Access to sample contracts (public government templates, open datasets, or partner law firms).
- Budget for cloud GPU credits and possible paid datasets.

### 3.4 Data: sources and collection plan

**Source types**

1. **Public datasets**: CUAD (Contract Understanding Atticus Dataset), ContractNLI and other open contract datasets for baseline techniques and bootstrapping.
2. **Public domain contract templates**: government & real-estate industry published templates (rental/lease/purchase forms).
3. **Redacted real contracts**: negotiate access with partner law firms or anonymize donated documents.
4. **Sri Lanka-specific documents**: government tenancy forms, sale deeds, and publicly available sample clauses (to ensure local language/terminology coverage).
5. **Synthetic augmentation**: generate clause variants with controlled bias using an LLM (carefully validated by experts) to augment scarce classes.

**Collection & privacy**

- All real contracts will be anonymized prior to annotation; PII redaction enforced. If partner data is used, sign an NDA or data-sharing agreement that specifies permitted use and retention policies. Store raw data encrypted and keep an audit log.

**Annotation design**

- Use the annotation guideline that defines: clause boundaries, bias categories, severity labeling rules, and bias-direction annotation.
- Pilot annotation on 200–500 clauses with 2–3 annotators per clause to measure inter-annotator agreement (Cohen's $\kappa$).
- Resolve disagreements via adjudication by a legal expert, then refine the guidelines.
- Use this adjudicated data as gold standard for evaluation and calibration.

**Expert review / user study**

- Get guidance from a legal expert for qualitative evaluation.

- Study design: Expert reviews a sample of 30–50 annotated documents with model outputs; metrics: perceived usefulness (Likert scale), time-to-review (measure), correctness of flagged clauses, and acceptability of trust_score.
- Ethical approval and consent forms should be obtained before user study.

### 3.5 Timeline & Task Chart

**Insert Figure 3.2: Gantt chart / Task schedule here.**

**Table 3.1: Task schedule**

| Phase | Task (subtasks) | Weeks |
|---|---|---|
| A | Design taxonomy, annotation guidelines, system design | 1–2 |
| B | Data collection, OCR, pilot annotation (guideline revision) | 3–8 |
| B | Full annotation and adjudication | 9–12 |
| C | Baselines + transformer fine-tuning + calibration | 13–16 |
| D | UI prototype & API integration | 17–19 |
| E | Evaluation: quantitative + expert qualitative study | 20–21 |
| E | Iteration & improvements from feedback | 22–23 |
| F | Final report, deployment guidelines & presentation | 24 |

### 3.6 Risk assessment & mitigation

- **Data scarcity for property clauses** — *Mitigation*: use public corpora to bootstrap, synthetic augmentation, and partnerships with law firms for redacted documents.
- **Low inter-annotator agreement** — *Mitigation*: refine guidelines, run iterative pilot annotations and adjudication sessions. Target $\kappa \geq 0.7$.

- **Model overconfidence / poor calibration** — *Mitigation*: apply calibration (temperature scaling) and evaluate Expected Calibration Error (ECE). Use human-in-loop for critical outputs.
- **Jurisdictional variation (terminology differences)** — *Mitigation*: include international and Sri Lankan examples; consider metadata selection by jurisdiction and train a domain adapter layer.
- **Ethical/legal misuse** — *Mitigation*: clearly label outputs as advisory, preserve audit logs, and provide deployment recommendations requiring human sign-off.

### 3.7 Anticipated conclusions and real-world use

**Anticipated technical results**

- A working prototype that segments clauses, predicts bias categories/direction and severity, and computes a calibrated trust_score.
- Quantitative results expected: competitive F1 scores on clause-level detection (target ≥0.70 for high-severity labels) and a well-calibrated trust_score (ECE target <0.10). Exact numbers will be refined after baseline experiments.

**Anticipated practical outcomes**

- Reduced time for initial contract triage for non-experts and legal reviewers by surfacing likely problematic clauses.
- A dataset and annotation guidelines that can bootstrap future research into property agreement bias.
- Recommendations for safe deployment (human-in-loop review, jurisdictional disclaimers, data governance).

**Limitations**

- The system is a support tool — not a substitute for legal advice. Performance depends on quality and breadth of annotated data and will vary across jurisdictions and contract styles.

# 4. Project Requirements

## 4.1 Functional requirements

1. **Document intake**
    - Accept uploads of DOCX, PDF (including scanned images → OCR), and TXT files and allow copy-paste of raw text.
    - Accept optional metadata at upload: jurisdiction, contract type (lease/rental/purchase), language.
2. **Text extraction & clause segmentation**
    - Extract text with character offsets for mapping annotations back to the original file.
    - Split document into clause-level units (clauses/paragraphs/numbered items) and provide surrounding context (±1 clause window).
3. **Preprocessing & entity extraction**
    - Normalize dates, monetary values and standardize units.
    - Run NER for parties, amounts, dates, durations, jurisdictions and label them for use in rules/explanations.
4. **Bias & risk classification**
    - For each clause produce: one or more **bias category labels**, **bias direction** (Owner / Buyer-Tenant / Neutral), **severity** (Low / Moderate / High) and a confidence score for each prediction.
5. **Trust_score computation**
    - Aggregate clause-level outputs into a calibrated **trust_score** in [0.0, 1.0] where **0.0** = high bias/risk and **1.0** = low bias/risk.
    - Provide provenance info indicating which clauses most influenced the trust_score.
6. **Explainability & recommendations**
    - For each flagged clause provide: (a) short plain-language rationale, (b) model confidence, (c) relevant extracted entities, (d) suggested remediation wording (template language) where applicable.

- Allow toggling between model-driven and rule-based rationale.

7. **Interactive UI**
    - Show original document with color-coded clause highlights (Low = yellow, Moderate = orange, High = red).
    - On click/hover open tooltip/side panel with rationale, labels, confidence and action buttons (Accept / Mark false positive / Suggest edit).
8. **Exports & reporting**
    - Export annotated document (DOCX/PDF) with highlights and an exportable summary report (CSV/JSON) that lists clauses, labels, confidences, and trust_score.
9. **Human feedback & retraining loop**
    - Record user corrections and feedback to store as labeled data for periodic model retraining.
    - Provide an admin interface to review feedback and approve examples for retraining.
10. **Audit & logging**
    - Maintain immutable audit logs for uploads, model inferences, user interactions and corrections for traceability.
11. **Access control & multi-user support**
    - Support multiple user roles (Admin, Legal Expert, Annotator, End User) with role-based permissions.
12. **API**
    - Provide REST API endpoints for: /upload, /analyze, /status/{job_id}, /feedback, /export, /health. API responses should include structured JSON with clause spans and label metadata.

### 4.2 Non-functional requirements

1. **Performance & latency**

- Inference latency target: ≤ 10 seconds for documents up to 20 pages (single request end-to-end: extraction → segmentation → inference → response).
- Batch/async processing allowed for very large documents with clear progress/status.

2. **Scalability**
    - System must scale horizontally for inference (stateless model-serving) and scale storage independently (object storage for documents).
    - Support concurrent users and workload spikes via container orchestration (Kubernetes).
3. **Reliability & availability**
    - Target uptime ≥ 99% during business hours; graceful degradation (read-only UI) if model service unavailable.
4. **Security & privacy**
    - TLS for all network communication.
    - Encryption at rest for stored documents and annotations.
    - PII redaction/preprocessing step to remove or mask personal data.
    - Role-based access control and strong authentication (OAuth2 / SSO support).
    - Data retention and deletion policies configurable (e.g., auto-delete after X days).
5. **Maintainability & extensibility**
    - Modular architecture so models, tokenizer, or rules can be swapped without UI overhaul.
    - Clear CI/CD pipelines for model and app releases; versioning for models and datasets.
6. **Explainability & transparency**
    - Provide human-readable rationales and confidence calibration; produce model version & dataset provenance with every result.

7. **Usability & accessibility**
    - Intuitive UI with keyboard navigation and screen-reader compatibility (WCAG AA recommended).
    - Clear legends, tooltips, and help/documentation inline.
8. **Auditability & compliance**
    - Comprehensive audit trails for legal/regulatory review and reproducibility of results.
    - Logs retained per policy and exportable for compliance audits.
9. **Localization**
    - Support English and ability to extend to other languages or regional legal terms (Sri Lanka specifics included).
10. **Testing**
    - Unit tests, integration tests, and performance/load tests included in delivery; model evaluation scripts reproducible.

**4.3 User requirements**

**User roles & primary needs**

1. **End User (Individual / Consumer)**
    - Simple upload flow and a clear overall trust_score with highlighted problematic clauses and short remediation suggestions.
    - Non-technical plain language, minimal legal jargon.
2. **Legal Reviewer / Lawyer**
    - Detailed clause views, full provenance, model rationale, ability to edit flagged text and export annotated documents.
    - Access to confidence scores and a control to mark or annotate model outputs.
3. **Annotator / Data Curator**
    - Annotation workspace with clause-level labeling interface, adjudication queue, and bulk export/import for annotation labelling tasks.

4. **Administrator / Project Lead**
    - Dashboard showing usage metrics, model performance over time, retraining triggers, user & access management, and dataset management.

**UX expectations**

- One-click upload + results page.
- Clear legend for severity colors and a persistent trust_score widget.
- Feedback buttons (e.g., “This is incorrect”, “Suggest wording”) on every flagged clause.
- Export and share features for legal counsel.

**Training & documentation**

- Short end-user manual and a more detailed admin + annotator guide.
- Onboarding session material for legal experts to interpret model outputs and use the feedback loop.
-

**4.4 System requirements**

**Hardware**

- **Development / Testing**
    - Dev machines (laptop/desktop) with 16 GB RAM, modern CPU.
- **Training**
    - GPU instance(s): minimum single NVIDIA T4 or V100 (16–32 GB VRAM) or equivalent cloud GPU (GCP/AWS/Azure).
- **Inference**
    - CPU instances for low-volume inference; GPU instances or multi-CPU scaled pods for low-latency SLAs.
- **Storage**
    - Object storage (S3/GCS) for uploaded docs and model artifacts; estimated project storage (start) 100 GB, scalable.

**Software stack & frameworks**

- Language: Python 3.9+
- ML: PyTorch or TensorFlow, Hugging Face Transformers
- Model serving: TorchServe / Triton / FastAPI + Uvicorn (containerized)
- Web UI: React or Next.js (frontend) + Flask / FastAPI (backend)
- Database: PostgreSQL (metadata, users) + Redis (caching / job queue)
- Annotation: Label-studio or Doccano for in-house labeling
- OCR / Extraction: Tesseract, Apache Tika, pdfminer.six
- Observability: Prometheus + Grafana; logging to ELK or cloud logging.
- CI/CD: GitHub Actions / GitLab CI; Docker; Kubernetes for orchestration.
- Security: OAuth2 / OpenID Connect provider (university SSO if available); TLS cert management.

**Integration & API**

- RESTful JSON APIs with versioning (v1, v2).
- Webhooks for job completion notifications.
- Authentication headers for each API call; RBAC enforced at API level.

**Data & model management**

- Model registry (MLflow or similar) with model metadata (version, training data snapshot, performance metrics).
- Dataset storage with immutable gold-standard partitions for evaluation.
- Backup policy: daily incremental backups and monthly full backups for critical data.

**Testing & quality assurance**

- Unit tests covering preprocessing, segmentation, and core logic.
- Integration tests for upload → OCR → segmentation → model inference → UI mapping.
- Performance tests simulating concurrent users and large documents.

- Security testing: vulnerability scanning and basic penetration testing before deployment.

**Acceptance criteria (high level)**

- System accepts DOCX/PDF/TXT and returns clause-level annotations and a trust_score.
- UI shows highlights and detailed rationale for flagged clauses.
- Quantitative model performance meets baseline F1 targets for high severity labels (e.g., F1 $\geq 0.70$) and trust_score calibration ECE $< 0.10$ (targets adjustable post-baseline).
- Audit logs capture 100% of inference events and user feedback.
- Role-based access controls enforce correct permissions for all user types.
- All sensitive data encrypted at rest and in transit; PII redaction implemented.

## References


[1] D. Hendrycks, C. Burns, A. Chen and S. Ball, “CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review,” *arXiv:2103.06268*, Mar. 2021.

[2] M. Lippi, P. Palka, G. Contissa, F. Lagioia, H.-W. Micklitz, G. Sartor and P. Torroni, “CLAUDETTE: an Automated Detector of Potentially Unfair Clauses in Online Terms of Service,” *arXiv:1805.01217*, May 2018.

[3] Y. Koreeda and C. D. Manning, “ContractNLI: A Dataset for Document-level Natural Language Inference for Contracts,” *Findings of EMNLP*, 2021.

[4] I. Chalkidis, S. Fergadiotis, A. Androutsopoulos et al., “LEGAL-BERT: The Muppets straight out of Law School,” *Findings of EMNLP*, 2020.

[5] I. Chalkidis, A. Jana, D. Hartung, M. Bommarito, I. Androutsopoulos, D. M. Katz and N. Aletras, “LexGLUE: A Benchmark Dataset for Legal Language Understanding in English,” *ACL / arXiv*, 2021/2022.

[6] D. Tuggener, P. von Däniken, T. Peetz, M. Cieliebak and M. (others), “LEDGAR: A Large-Scale Multi-label Corpus for Text Classification of Legal Provisions in Contracts,” *Proc. LREC*, 2020.

[7] Council Directive 93/13/EEC of 5 April 1993, “On unfair terms in consumer contracts,” *Official Journal L 95/29*, Apr. 1993.

[8] I. Chalkidis, M. Fergadiotis, N. A. (et al.), “Processing Long Legal Documents with Pre-trained Transformers,” *Proceedings / ACL Workshop on Legal NLP (LexGLUE related work)*, 2022.

[9] (Recent) “Evaluating Legal-Specific LLMs on Contract Understanding,” *arXiv preprint*, 2025 — (analysis and benchmarks of legal LLMs on contract tasks).

[10] A. Tuggener, P. von Däniken, T. Peetz et al., “LEDGAR: A Large-Scale Multi-label Corpus for Text Classification of Legal Provisions in Contracts,” *LREC*, 2020.

[11] A. Vaswani, N. Shazeer, N. Parmar et al., “Attention is All You Need,” *Proc. NeurIPS*, 2017.

# Appendices